gemini-conversation

Exported on: 2026/3/17 23:01:28

User

Dockerコンテナで動かす前提でgoのcliアプリのhello world的なGetting Startedを教えてください。

Gemini

Dockerでの実行を前提としたGo製CLIツールの「Hello World」、非常に良いスタートですね！

Goはコンパイル後に単一のバイナリになるため、Dockerとの相性は抜群です。今回は、**「開発用のコード」から「軽量な実行用イメージ」**を作るまでの標準的な流れを解説します。

---

1. プロジェクトの準備

まずはローカル環境でディレクトリを作成し、Goモジュールを初期化します。（Goがインストールされていなくても、後のDocker工程で完結できますが、ファイル準備だけ行います）

1. ディレクトリ作成: `mkdir go-docker-cli && cd go-docker-cli`
2. モジュール初期化: `go mod init go-docker-cli`

メインプログラムの作成（`main.go`）

標準入力から名前を受け取り、挨拶を返すシンプルなCLIプログラムを書きます。

Go

```go
package main

import (
	"fmt"
	"os"
)

func main() {
	name := "World"
	if len(os.Args) > 1 {
		name = os.Args[1]
	}
	fmt.Printf("Hello, %s! (Running inside Docker)\n", name)
}
```

---

2. Dockerfileの作成

ここが肝心です。**「マルチステージビルド」**という手法を使い、ビルド環境と実行環境を分けることで、最終的なイメージサイズを劇的に小さくします。

`Dockerfile` という名前で以下のファイルを保存してください。

Dockerfile

```
# --- ステージ1: ビルド環境 ---
FROM golang:1.24-alpine AS builder

# 作業ディレクトリの設定
```

```
WORKDIR /app

# 依存関係のコピーとダウンロード
COPY go.mod ./
RUN go mod download

# ソースコードをコピーしてビルド
COPY . .
RUN go build -o /hello-cli

# --- ステージ2: 実行環境 ---
FROM alpine:latest

# ビルドしたバイナリだけをコピー
COPY --from=builder /hello-cli /hello-cli

# 実行コマンドの設定
ENTRYPOINT ["/hello-cli"]
```

---

3. ビルドと実行

それでは、実際にイメージを作って動かしてみましょう。

イメージのビルド

Bash

```
docker build -t my-go-app .
```

コンテナの実行

引数なしで実行する場合：

Bash

```
docker run --rm my-go-app
# 出力: Hello, World! (Running inside Docker)
```

引数を渡して実行する場合（CLIアプリとしての動作確認）：

Bash

```
docker run --rm my-go-app Gemini
# 出力: Hello, Gemini! (Running inside Docker)
```

Tips: なぜこの構成が良いのか？

- **マルチステージビルド:** `golang` イメージ（約300MB〜）でビルドし、実行には `alpine` （約5MB）を使うことで、配布用イメージを最小限に抑えられます。
- **ENTRYPOINT:** これを使うことで、 `docker run my-go-app [引数]` のように、コンテナ自体を一つのコマンドのように扱えるようになります。

次は、このアプリをより本格的にするために、**「Goの外部ライブラリ（Cobraなど）を導入して、より複雑なコマンドライン引数を処理する方法」**について解説しましょうか？

User

cobraを入れるメリットを教えてください。

Gemini

GoでCLIアプリを作る際、標準ライブラリの `os.Args` や `flag` パッケージだけでも動きますが、**Cobra** を導入すると「本格的なツール」へのハードルが一気に下がります。

Cobraは、Docker、Kubernetes (kubectl)、GitHub CLI (gh) など、名だたるツールで採用されているデファクトスタンダードです。

Cobraを導入する4つの主なメリット

1. サブコマンドの構造化が容易

`docker run` や `docker build` のように、一つのツールに複数の機能（サブコマンド）を持たせるのが非常に簡単になります。

- **標準ライブラリ:** `if` 文や `switch` 文で `os.Args[1]` を自力で判定する必要がある。
- **Cobra:** `cobra add [command]` で独立したファイルとして機能を追加でき、コードの通しが良くなる。

2. 高度なフラグ（オプション）管理

「短い名前 (`-v`)」と「長い名前 (`--verbose`)」の両方に対応したり、特定の型（int, string, slice等）を自動でパースしたりできます。

- 必須フラグの設定や、デフォルト値の設定も数行で書けます。
- グローバルフラグ（全コマンド共通）とローカルフラグ（特定のサブコマンドのみ）の使い分けも明確です。

3. 自動生成されるヘルプと補完

ここが最大の魅力かもしれません。

- **ヘルプ画面:** `-h` や `--help` を入力した際の綺麗なマニュアルを自動で生成します。
- **シェル補完:** Bash, Zsh, Fish などのタブ補完スクリプトを自動生成できるため、ユーザー体験が劇的に向上します。

4. ボイラープレート（雛形）の自動作成

`cobra-cli` というツールを使えば、プロジェクトの初期構成を一瞬で作ってくれます。ディレクトリ構成に悩む必要がなくなります。

構造のイメージ

Cobraを使うと、CLIアプリは以下のようなツリー構造で管理されるようになります。

実際にCobraをプロジェクトに追加してみる？

今の「Hello World」プロジェクトにCobraを導入する手順は以下の通りです。

1. **ツールのインストール:** `go install github.com/spf13/cobra-cli@latest`
2. **初期化:** `cobra-cli init`
3. **コマンドの追加:** `cobra-cli add hello`

これを実行すると、自動的に `cmd/root.go` や `cmd/hello.go` が作成されます。

「実際にCobraを使って、引数によって挙動が変わる本格的なCLIコマンドを作ってみる手順」 を解説しましょうか？

User

mycli video generate (first-frame) (last-frame)とかmycli image generate (input-image) (prompt)みたいなモジュール/コマンド/複数パラメータみたいな最終イメージで考えてるんですが、いきなりcobra入れて始めちゃったほうが良さそう？

Gemini

結論から言うと、**「いきなりCobraを入れて始める」ことを強くおすすめします。**

その構成（サブコマンドの階層化と複数パラメータ）を標準ライブラリだけで実装しようとすると、`os.Args` のパースやバリデーション自作に時間を取られ、本来作りたい機能（画像・動画生成）に集中できなくなるからです。

なぜその構成にCobraが「必須」レベルなのか

お考えの `mycli video generate` という構造は、Cobraでは以下のように整理されます。

1. 階層構造（Command Tree）の管理

Cobraはコマンドを「木構造」で管理します。

- `mycli` (Root)
  - `video` (Subcommand)
    - `generate` (Sub-Subcommand)
  - `image` (Subcommand)
    - `generate` (Sub-Subcommand)

標準ライブラリでは「2番目の引数が `video` で、3番目が `generate` なら...」というネストした `switch` 文地獄になりますが、Cobraなら各コマンドを独立した関数として定義できます。

2. 位置引数（Args）のバリデーション

`video generate (first-frame) (last-frame)` のように「必ず2つの引数が必要」という制約も、Cobraなら1行で書けます。

Go

```go
Args: cobra.ExactArgs(2), // 引数が2つ以外なら自動でエラーメッセージを出して終了
```

3. フラグ（Options）との組み合わせ

将来的に `mycli video generate --fps 30` のようなオプションを追加したくなった際も、既存の構造を壊さずに数行追加するだけで対応可能です。

爆速で始めるステップ（Docker前提）

今のプロジェクトを一旦忘れて、以下の手順で「骨組み」を作るのが一番近道です。

1. **Cobra-cliをインストール（ローカルまたはコンテナ内）**

   Bash

   ```bash
   go install github.com/spf13/cobra-cli@latest
   ```

2. **プロジェクト初期化**

Bash

```bash
mkdir mycli && cd mycli
go mod init mycli
cobra-cli init
```

3. **コマンドの追加（階層を作る）**

Bash

```bash
# videoコマンドと、その下のgenerateを追加
cobra-cli add video
cobra-cli add generate -p 'videoCmd'  # videoの子として追加

# imageコマンドと、その下のgenerateを追加
cobra-cli add image
cobra-cli add generate -p 'imageCmd'  # imageの子として追加
```

これで、`cmd/video.go` や `cmd/generate.go` が自動生成されます。

次のステップへの提案

Goのコード内で実際に動画や画像を生成する際、背後で別のAPIを叩いたり、重い処理を走らせたりすることになると思います。

「Cobraで生成された `cmd/generate.go` の中に、どうやって実際のロジックを書き込んでいくか（具体的なコード例）」 を見てみますか？

User

実際のロジックはすでにバックエンドのREST/socket.ioサーバーが有るのでそれを呼び出すだけです。基本的に長時間かかるAI生成や双方向対話プロセスが多く、ユーザーはGUIを使う前提なので、cliはユーザーが直接PCでつかうというよりAIエージェントがコマンドごとにサブプロセスで実行するイメージです。jwt認証が必須ですが、これもcliでログイン処理するというよりは、cliが動くコンテナ内に仕込まれた環境変数からトークンなどをコマンド実行時に読み込んでhttpヘッダに加えてリクエストを投げるイメージです。
開発を始めるにあたってアドバイスや有益な知見があれば教えてください。

Gemini

AIエージェントがサブプロセスとして実行し、バックエンド（REST/Socket.io）を叩くブリッジとしてのCLIですね。非常に明確なアーキテクチャです。

その構成で開発を進めるにあたって、**「対マシン（AI）」および「コンテナ環境」**を意識した、Go + Cobraでの実装アドバイスを整理しました。

---

1. エラーハンドリングと終了コードの徹底

AIエージェントがサブプロセスとして実行する場合、AIは「標準出力（stdout）」と「終了コード（Exit Code）」で成否を判断します。

- **stderrの活用:** 正常なレスポンス（生成されたURLやID）は `stdout` に、デバッグ情報やエラーメッセージは必ず `stderr` に出すように徹底してください。
- **終了コードの細分化:** 単なる `exit 1` ではなく、`1: 認証エラー`, `2: ネットワークエラー`, `3: タイムアウト` のように分けると、AIエージェント側で「リトライすべきか」の判断が容易になります。

2. JSON出力モードの搭載

AIエージェントが結果をパースしやすいよう、`--json` フラグをグローバルに持たせることを強く推奨します。

Go

```go
// 理想的な出力イメージ
{
  "status": "processing",
  "job_id": "video-123",
  "estimated_seconds": 30
}
```

Goでは構造体を `json.Marshal` するだけなので実装負荷は低いです。

3. Socket.io (Websocket) の扱い

GoでSocket.ioクライアントを実装する場合、`googollee/go-socket.io` などのライブラリがありますが、CLI（サブプロセス）として動かすなら以下の点に注意が必要です。

- **コンテキストの制御:** AIエージェントがサブプロセスを `SIGTERM` で落とした際、ソケットを綺麗に閉じるために `context.Context` を使った終了処理（Graceful Shutdown）を組み込んでおきましょう。
- **ステータス同期:** 長時間処理の場合、Socket.ioのイベントをそのまま `stdout` に一行ずつプログレスとして流すと、AI側が進捗を監視しやすくなります。

4. 認証情報の「環境変数」管理

「コンテナ内の環境変数からJWTを読み込む」設計は、Goの `viper` （Cobraと同じ作者のライブラリ）を使うと非常にスマートに書けます。

Go

```go
// cmd/root.go 等で設定
viper.AutomaticEnv() // システムの環境変数を自動読み込み
token := viper.GetString("AUTH_TOKEN") // $AUTH_TOKEN を取得
```

5. 開発効率を上げる構成案

複数のパラメータ（first-frame, prompt等）を扱うため、以下のようなフォルダ構成にすると、ロジックとCLIインターフェースを分離でき、テストがしやすくなります。

Plaintext

```
.
├── cmd/            # Cobraのコマンド定義（入力のバリデーションのみ）
│   ├── root.go
│   ├── video.go
│   └── image.go
├── internal/       # 外部から参照されないロジック
│   ├── api/         # RESTクライアント (http.Client)
│   ├── socket/      # Socket.ioクライアント
│   └── config/      # 環境変数やJWTの管理
├── Dockerfile
└── main.go
```

次の具体的なステップとして...

まずは、**「環境変数からJWTを読み込み、REST APIのヘッダーにセットして、単純なGETリクエストを投げる共通クライアント」**の部分を作ってみるのはいかがでしょうか？

もしよろしければ、**「Goでの標準的なHTTPクライアントの実装例（JWTヘッダー付与版）」**のコードサンプルを提示しましょうか？